

Course Group Project

Reinforcement Learning

DSOR 646

Assigned: 13 May 2026

Due: 1100, 10 June 2026

Each student should form a group of no more than three students. Each group must complete a course project involving the specification and implementation of three reinforcement learning (RL) algorithms to solve the sequential decision problem faced by the agent within the *LunarLander-v2* (https://gymnasium.farama.org/environments/box2d/lunar_lander/) Farama Foundation Gymnasium Environment. Gymnasium is a suite of RL problems originally created by Open AI as the Gym Environment [1] to test RL solution approaches.

Submission: Please turn in a printed copy of your report. Also submit your report and supporting Python files via an online submission.

Project Directions

Evaluate and compare three of the following five RL algorithms via rigorous computational testing. *You must encode your own implementation of the algorithms in Python.*

1. Semi-gradient n -step SARSA with linear VFA (p.247)
2. Least Squares Policy Iteration (LSPI; Lagoudakis & Parr [2])
3. SARSA(λ) with linear VFA (p.305)
4. REINFORCE with Baseline using linear VFA (p. 330)
5. Actor-Critic with Eligibility Traces (AC(λ)) using linear VFA (p. 332)

For each of your selected algorithms, perform testing with respect to:

1. **Effectiveness**
 - **Algorithm Score (AlgScore):** As in previous assignments, use a 60/40 weighted composite score of the 95% lower confidence bounds for Mean Max EETDR (peak) and Mean Time-Avg EETDR (sustained). Use 10 replications with 500–2000 episodes per replication. It may be prudent to conduct a second, more narrowly tailored, round of tuning experiments to further improve results.
 - **Superlative Policy (SP95LB):** The 95% CI lower bound of your overall best policy. Evaluate this policy using 100 independent test episodes (with different random seeds than milestone testing).
2. **Efficiency:** Time expended to execute testing.
3. **Robustness:** Ability to easily tune algorithms (i.e., how effective were design runs across the *entire region* of hyperparameter experimentation? Is an algorithm implementation “fragile” or overly sensitive to input?)

Perform hyperparameter tuning using a parallelized Latin Hypercube Design (LHD). Tune the parameters relevant to your chosen algorithms, such as:

- **Learning & Exploration:** Initial rates (α_a, ϵ_a) and polynomial decay parameters (α_b, ϵ_b).

- **Bootstrapping & Traces:** n -step horizon, or initial eligibility trace (λ_a) and decay (λ_b).
- **Linear VFA Parameters:** Number of tilings, tiles per tiling, Fourier cosine features, or polynomial orders.

Perform an OLS regression ANOVA, reporting the impact of each hyperparameter on Algorithm Score. Ensure you discuss the adjusted R^2 of overall fit and the sign, magnitude, and p -values of OLS coefficients for impactful hyperparameters.

Deliverables & Presentation

The course project is worth 30% of the total grade for this course. A 20-minute project presentation must be given during the final exam period (1100-1400, 10 Jun 2026). A printed project report is due at the start of the final exam period. Also upload the report as a pdf file and supporting Python files.

The project presentation should focus on your results and analysis, including a direct comparison of algorithms. Each student must speak. Video evidence of policy performance is welcome if it supports your narrative. RL presentations often tell a crawl-walk-run narrative, showcasing how the agent learned the task.

Required Report Structure

1. **Problem description:** A narrative explaining the important problem features, background, and context as modeled by the environment.
2. **Markov decision process model:** Describe the sequential decision problem **using mathematical notation:**
 - (a) State variable, $s \in \mathcal{S}$
 - (b) Action variable, $a \in \mathcal{A}$
 - (c) Reward function, $r(s, a, s')$
 - (d) State transition function, $S^M(s, a, w)$. Note that the state transition function (i.e., system model) $S^M(s, a, w) = s'$ returns the next state as a function of the current state s , current action a , and the random variate w , a realization of the (possibly state- and action-dependent) joint random variable driving stochastic evolution of the system forward in time.
 - (e) Objective
3. **RL Solution Approaches:** Describe your selected RL solution approaches, including algorithm pseudo-code. Describe important algorithm design features, including (as needed):
 - (a) Value function approximation (VFA) strategy (Basis functions, Tile coding scheme)
 - (b) Smoothing rule for PFA and/or VFA model coefficients
 - (c) Exploration-exploitation strategies
 - (d) Other algorithm features, depending on your approach
4. **Results and Analysis:**
 - (a) Describe your analysis approach, designed to evaluate the performance of your RL solution procedures with respect to quality of solution, computational effort, and robustness.

- (b) Experimentation should be used to determine the best hyperparameter values for your RL algorithms as well as the impact of the hyperparameters on the Algorithm Score. Indicate importance of algorithm design features. How sensitive is your algorithm implementation to its hyperparameters?
- (c) **Tables & Figures:**
 - Report a **Top 10 table** indicating best hyperparameter design runs, rank ordered by AlgScore, including component statistics and SP95LB. From which top scoring design run does the overall superlative policy derive?
 - Report a **Top 5 table** indicating best performing policies (from the final 100-episode evaluation), rank ordered by SP95LB.
 - Display a **learning curve** showing the best hyperparameter design run.
 - Compare algorithms: *Learning curves* for top performing hyperparameter design runs for each algorithm should be displayed on a single figure so they may be directly compared. Ensure consistent scales between figures. Other interesting visual aids are welcome.
- (d) Provide comparative insights given your results. Include an executive summary results table (e.g., best design run performance results and attendant statistics for each algorithm, as well as superlative policy results). Overall, how did your algorithms perform? Which algorithm would you recommend? Why?
- (e) **Advanced Strategies (Optional but encouraged):** Describe your implementation and results if you used more sophisticated RL design strategies discussed in class, such as:
 - *Reward shaping*: Giving additional bonuses and penalties extraneous to the actual RL problem during training to improve learning. Make sure to anneal (decay) such shaping over time, and evaluate milestone and final policies without reward shaping elements.
 - *Expert heuristic guidance*: Selecting some actions according to a fixed benchmark policy to encourage emulation.
 - Note: These strategies introduce bias into learning, but such bias can accelerate learning to attain initial good results to improve upon.

5. Conclusion:

- Key takeaways
- Next steps if you had more time
- Comment on the ease of algorithm implementation and the totality of the analysis experience. What was the most challenging aspect of the project? What was the most rewarding?

6. Division of labor: Report the division of labor among your group.

7. References

8. Appendices: Include a generative artificial intelligence (genAI) discussion and the Python implementations of your selected algorithms.

GenAI Usage: You may use genAI tools to assist you during this assignment. You may access Google's Gemini 3.1 Pro on genAI.mil or use another genAI model of your choice. Include an appendix describing which genAI models were used, an example of specific prompts written, and a reflection on how the model's outputs were incorporated. You are ultimately responsible for the correctness and completeness of all submitted work.

References

- [1] Brockman, Greg, Cheung, Vicki, Pettersson, Ludwig, Schneider, Jonas, Schulman, John, Tang, Jie, & Zaremba, Wojciech. 2016. OpenAI Gym. *arXiv preprint arXiv:1606.01540*.
- [2] Lagoudakis, Michail G, & Parr, Ronald. 2003. Least-squares policy iteration. *The Journal of Machine Learning Research*, 4, 1107–1149.